

第十章：SBI 固件与启动接口

一、核心概念

1.1 SBI (Supervisor Binary Interface)

SBI 是 RISC-V 架构中 M 态固件为 S 态内核提供的标准服务接口，类比其他架构的固件接口：

架构	固件接口	运行模式	提供的服务
RISC-V	SBI (OpenSBI/RustSBI)	M-mode	定时器、IPI、关机、串口
x86	UEFI Runtime Services	Ring 0 (特殊)	时间、变量存储、重启
ARM	PSCI (Power State Coordination Interface)	EL3	CPU 电源管理、多核启动

1.2 RISC-V 特权级与中断委托

M-mode (Machine)

← OpenSBI, 最高权限

↕ ecall / mret

S-mode (Supervisor)

← 内核

↕ ecall / sret

U-mode (User)

← 用户程序

中断委托 (`medeleg/mideleg`)：OpenSBI 在初始化时将大部分异常和中断委托给 S 态，内核可以直接处理。只有少数操作（定时器、IPI）需要回调 M 态。

1.3 ecall 调用约定

SBI v0.2+ 规范：

a7 = Extension ID (EID)

a6 = Function ID (FID)

a0-a5 = 参数

ecall

a0 = error code

a1 = return value

1.4 关键 SBI 扩展

EID	扩展名	功能
0x00	Timer	set_timer
0x01	Console Putchar	调试输出
0x08	System Shutdown	关机

EID	扩展名	功能
0x48534D	HSM (Hart State Management)	多核启动/关闭
0x735049	IPI	核间中断

二、基本推论

推论 1：SBI 是 RISC-V 的"运行时固件服务"。 就像 UEFI 的 Runtime Services 在 OS 运行时仍然可用，SBI 在内核运行期间提供定时器、关机等服务。

推论 2：中断委托让内核不需要处理 M 态逻辑。 没有委托的话，每个中断都要先进 M 态再转发给 S 态，增加一次特权级切换的开销。

推论 3：定时器中断必须经过 M 态中转。 CLINT 定时器属于 M 态资源，S 态不能直接操作。定时器到期 → M 态中断 → OpenSBI 设 sip.SSIP → mret → S 态感知软件中断。

三、Linux / 通用实现

3.1 UEFI 启动阶段

x86 服务器的启动流程：

```
SEC (Security) → PEI (Pre-EFI Init) → DXE (Driver Execution)
→ BDS (Boot Device Selection) → OS Loader → 内核
```

Runtime Services： 操作系统启动后仍可用的 UEFI 服务：

- `GetTime / SetTime`
- `GetVariable / SetVariable` (NVRAM 变量)
- `ResetSystem` (重启/关机)

3.2 ARM 的 ATF + PSCI

ARM Trusted Firmware 提供：

- **PSCI：** CPU 电源管理 (CPU_ON/CPU_OFF/SYSTEM_RESET)
- **Secure Monitor：** EL3 异常处理
- **TrustZone 切换：** Normal World ↔ Secure World

3.3 Linux 的 SBI 调用

Linux 内核通过 `sbi_ecall()` 封装所有 SBI 调用：

```
struct sbiret sbi_ecall(int ext, int fid, ...);
// sbi_set_timer() → sbi_ecall(SBI_EXT_TIME, 0, stime_value)
// sbi_send_ipi() → sbi_ecall(SBI_EXT_IPI, 0, hart_mask)
```

四、RUOS 的实现

核心文件

- `src/boot/entry.S` — 从 OpenSBI 接收控制权
- `src/boot/start.c` — `ecall` 包装函数

4.1 使用的 SBI 调用

SBI 调用	用途	调用点
<code>sbi_set_timer(val)</code>	设置下次定时器中断	时钟中断处理
<code>sbi_shutdown()</code>	关机	panic / 测试完成
<code>sbi_console_putchar(c)</code>	串口输出	早期 printf

```
static inline void sbi_set_timer(uint64 val) {
    register uint64 a0 asm("a0") = val;
    register uint64 a7 asm("a7") = 0x00; // Timer extension
    register uint64 a6 asm("a6") = 0x00;
    asm volatile("ecall" : "+r"(a0) : "r"(a7), "r"(a6));
}
```

4.2 定时器中断完整路径

```
1. sbi_set_timer(current_time + interval)
2. CLINT 触发 M 态中断
3. OpenSBI: 设 sip.STIP → mret
4. S 态感知 STIP → usertrap/kerneltrap → devintr()
5. devintr(): sbi_set_timer() 设下一次 + yield()
```

4.3 从 OpenSBI 到内核的交接

```
OpenSBI (M-mode, 0x80000000)
├── medeleg: 委托大部分异常给 S 态
├── mideleg: 委托外部/软件中断给 S 态
├── PMP: 配置物理内存保护
├── mepc = 0x80200000
├── mret → S-mode
└──
    ↓
_entry (a0=hartid, a1=FDT)
```

五、八股 / 面试高频问题

Q: RISC-V 有几个特权级？各自的作用？ A: 三个。M (Machine) = 最高权限，运行固件；S (Supervisor) = 运行 OS 内核；U (User) = 运行用户程序。M 态可以访问所有 CSR，S 态只能访问 s 前缀的 CSR。

Q: 什么是中断委托？为什么需要？ A: `medeleg/mideleg` 寄存器将指定的异常/中断直接交给 S 态处理，避免每次都先进 M 态再转发。没有委托，每个 trap 多一次 M→S 切换开销。

Q: UEFI 和 BIOS 的区别？ A: BIOS 是 16 位实模式，通过 INT 中断调用，无驱动框架。UEFI 是 32/64 位保护模式，有驱动模型 (DXE)，支持 GPT 分区、Secure Boot、网络启动。UEFI 是 BIOS 的现代替代。

Q: 类比 SBI 和 UEFI 的关系？ A: SBI 之于 RISC-V 内核，如同 UEFI Runtime Services 之于 x86 内核——都是固件在 OS 运行时仍提供的底层服务接口（定时器、关机等）。

六、项目贡献亮点

- 通过 SBI ecall 与 OpenSBI 固件交互，理解 M/S 态特权级切换和中断委托
- 理解定时器中断的 M→S 态传递路径 (CLINT → OpenSBI → SSIP → 内核)
- 可类比讲述 UEFI Runtime Services 的设计思路

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
M 态代码	完全不写	同 (依赖 OpenSBI)	教学简化
SBI 调用	直接 inline asm	sbi_ecall 统一封装	简单直接
定时器	SBI ecall	同 (RISC-V 标准做法)	无法直接操作 CLINT
IPI	未实现	sbi_send_ipi	2 核下暂不需要